

Browser and Application-level Caching

Browser-Level Caching (Client-side caching)

Browser caching stores resources locally so users don't have to re-download them on subsequent visits.

What it caches

- Images
- CSS/JS files
- API responses (optional)
- Fonts
- Static assets

How it works: Server sends headers telling browser how long data is valid.

Key HTTP Headers:

Cache-Control is the primary mechanism:

- max-age=3600 - Cache for 3600 seconds
- no-cache - Must revalidate with server before using cached version
- no-store - Never cache (sensitive data)
- public - Can be cached by browsers and CDNs
- private - Only browser can cache, not CDNs
- immutable - Resource never changes (great for versioned assets)

ETag (Entity Tag) enables conditional requests:

- Server sends: ETag: "abc123"
- Browser sends back: If-None-Match: "abc123"
- Server responds with 304 Not Modified if unchanged, or 200 with new content

Last-Modified is the older alternative:

- Server sends: Last-Modified: Wed, 21 Oct 2015 07:28:00 GMT
- Browser sends: If-Modified-Since: Wed, 21 Oct 2015 07:28:00 GMT

Common Strategies:

HTML files: Cache-Control: no-cache (always check for updates)

CSS/JS with version hashes: Cache-Control: public, max-age=31536000, immutable

Images: Cache-Control: public, max-age=86400

API responses: Cache-Control: private, max-age=300

Application-Level Caching (Server-side caching)

This involves caching data in your backend systems to reduce database load and improve response times.

Client → API → Cache → DB

1. In-Memory Caching (Redis, Memcached)

Most common pattern for hot data:

- Store frequently accessed data in RAM
- Sub-millisecond access times

- Redis supports data structures (strings, lists, sets, hashes)
- Memcached is simpler, just key-value

Typical use cases:

- User sessions
- Database query results
- API responses
- Rate limiting counters
- Leaderboards

2. Cache Patterns

Cache-Aside (Lazy Loading):

1. Application checks cache first
2. If miss, query database
3. Write result to cache
4. Return to user

Pros: Only cache what's needed

Cons: First request always slow (cache miss)

Write-Through:

1. Application writes to cache
2. Cache writes to database synchronously
3. Return to user

Pros: Cache always consistent

Cons: Write latency increased

Write-Behind (Write-Back):

1. Application writes to cache
2. Cache acknowledges immediately
3. Cache writes to database asynchronously
4. Return to user

Pros: Fast writes

Cons: Risk of data loss if cache fails

Read-Through:

Cache sits between app and database, automatically fetches on miss.

Cache invalidation strategies:

1. TTL (Time-to-live)

Auto expiration. Simple but may serve stale data.

2. Manual invalidation

3. **Versioned keys** - Changing version invalidates old cache.

4. Event-driven invalidation (advanced)

Cache problems interviewers expect

Cache stampede (thundering herd)

Many requests hit DB when cache expires simultaneously.

Solutions:

- Randomized TTL
 - Request coalescing
 - Mutex/locking
 - Background refresh
-

Cache inconsistency

Cache has stale data.

Tradeoff:

“We prefer eventual consistency for performance.”

Hot keys

One key gets massive traffic.

Solutions:

- Replicate cache
 - Partition keys
 - Local in-memory cache
-

Cold start

Empty cache after restart → DB overload.

Solutions:

- Warmup scripts
- Preloading popular data